Republic of the Philippines
BICOL UNIVERSITY POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay

WEB SYSTEMS AND TECHNOLOGIES

PHASE 1: FOUNDATION OF FINAL PROJECT

TASK MANAGEMENT SYSTEM

Prepared by:

Balauro, Christian A.

Matias, Kar James M.

Pabilona, Bea Roshan O.

Reantaso, Eleanor Aimee O.

Singh, Simranjit S.

Group 5

BSIS-2B

Submitted to:
GUILLERMO JR. V. RED
Instructor
Subject Instructor

October 2026

Project Proposal

Project Title

Task Management System: A Web-Based Application for Organizing Tasks and Deadlines

Introduction

Managing school activities can sometimes be difficult, especially when several assignments, projects, and deadlines need to be completed at the same time. Students may forget important tasks or have difficulty keeping track of their progress when they rely only on notebooks, messages, or memory. Because of this, our group came up with the idea of developing a Task Management System that can help users organize their responsibilities in one place. The system will provide a simple way to record tasks, check deadlines, and monitor which activities are already completed or still need attention.

System Purpose

The purpose of the Task Management System is to make managing daily tasks and school requirements more convenient. It will allow users to add tasks, write descriptions, set due dates and priorities, and update their progress as they work. Through its dashboard, users can easily review their pending, ongoing, and completed tasks. Our group also aims to apply what we have learned in Web Systems and Technologies by developing a functional web application using HTML, CSS, Bootstrap, JavaScript, Node.js, Express, MongoDB, and Mongoose.

Target Users

- Students who need to track assignments, projects, and deadlines.
- Individuals who want to organize personal responsibilities and daily activities.
- Administrators who need to manage accounts and oversee appropriate system activity.

Objectives

- Provide a simple, understandable interface for recording and managing tasks.
- Help users prioritize work and monitor task progress and deadlines.
- Store user and task information in a database so it can be retrieved and updated.

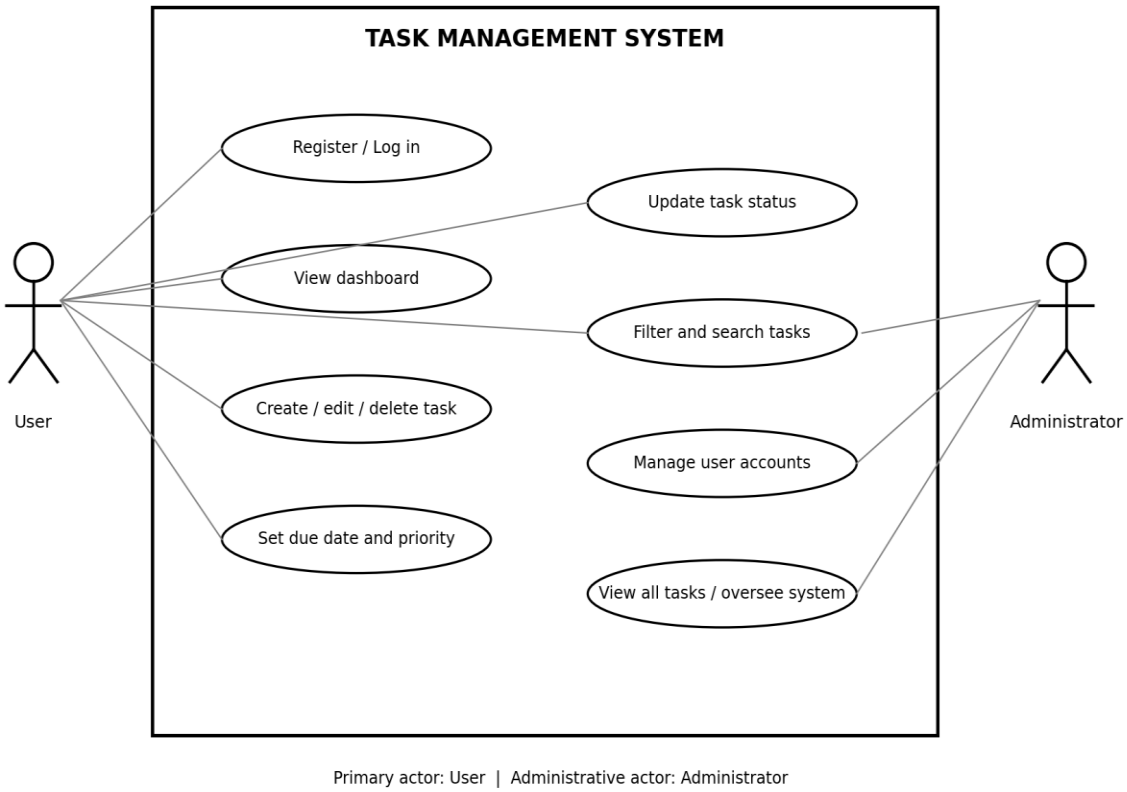
- Practice full-stack development, database modeling, version control, and teamwork.

Key Features

- User registration and login with protected account information.
- Create, view, edit, and delete personal tasks.
- Set a task title, description, due date, priority, and status.
- Filter or search tasks by status, priority, or keyword.
- Dashboard showing tasks and their current status.
- Administrator account-management and basic oversight features.

Use Case Diagram (UML)

The diagram shows the main actions available to a regular user and an administrator. A regular user manages their own tasks, while an administrator has additional account-management and oversight responsibilities.



Primary actor: User | Administrative actor: Administrator

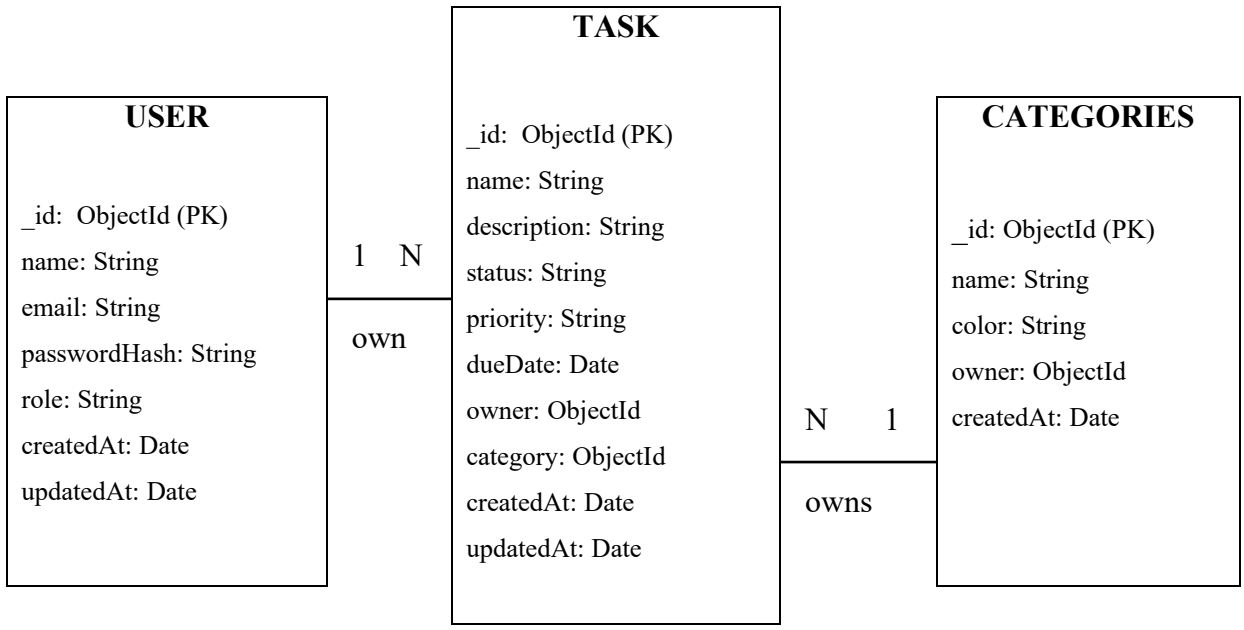
Use Case Notes

- User: registers or logs in, views the dashboard, creates/edits/deletes tasks, sets due dates and priorities, updates status, and searches or filters tasks.
- Administrator: manages user accounts and oversees system activity. Administrative actions should be restricted to authorized accounts.
- Security rule: a regular user must only access or modify their own tasks; the backend must verify ownership on every task operation.

The Use Case Diagram (UML) was created using draw.io (diagrams.net) to illustrate the interactions between users and the Task Management System, including the main functions that users can perform within the application.

Entity-Relationship Diagram (MongoDB Style)

The proposed design uses three collections: Users, Tasks, and Categories. MongoDB stores documents in collections; ObjectId references connect documents where needed. The category relationship is optional so tasks can still exist without a category.



Conceptual MongoDB model: each box represents a collection; ObjectId references connect related documents.

Collection and Field Definitions

Collection	Field	Data type	Notes
Users	_id	ObjectId	Primary identifier
Users	name	String	User's display name
Users	email	String	Required; unique; used for login
Users	passwordHash	String	Store a hash, never a plain-text password
Users	role	String	Allowed values: user or admin
Users	createdAt / updatedAt	Date	Managed timestamps
Tasks	_id	ObjectId	Primary identifier
Tasks	title	String	Required task title
Tasks	description	String	Optional details
Tasks	status	String	To Do, In Progress, Completed
Tasks	priority	String	Low, Medium, High
Tasks	dueDate	Date	Optional deadline
Tasks	owner	ObjectId	Reference to Users._id; required
Tasks	category	ObjectId	Optional reference to Categories._id
Tasks	createdAt / updatedAt	Date	Managed timestamps
Categories	_id	ObjectId	Primary identifier
Categories	name	String	Category label
Categories	color	String	Optional display color
Categories	owner	ObjectId	Reference to the user who owns the category
Categories	createdAt	Date	Creation timestamp

Relationships and Constraints

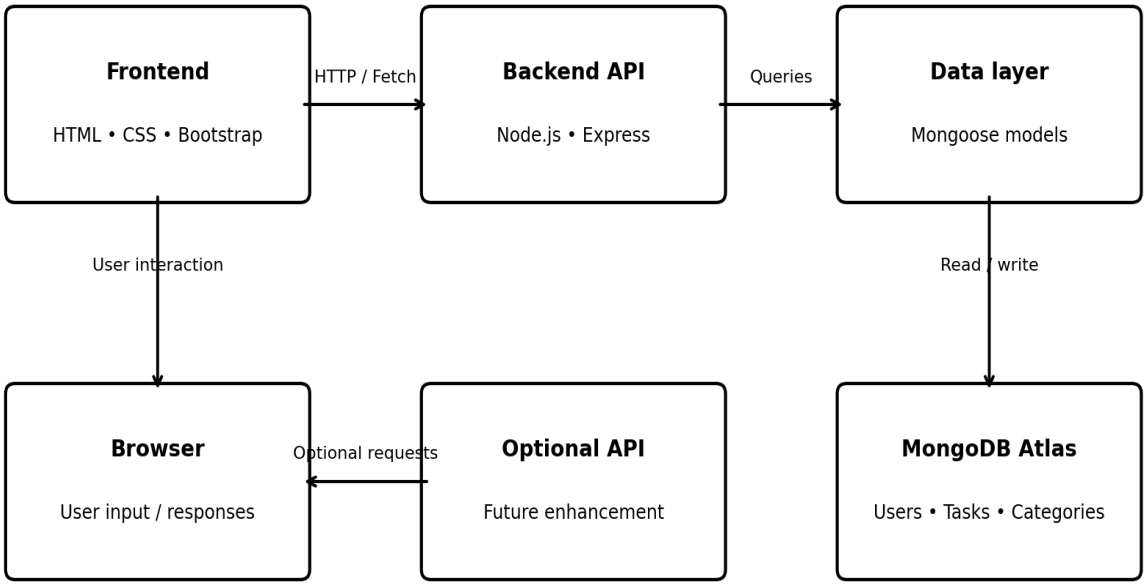
- One User can own many Tasks (one-to-many). Each Task has one owner.
- One User can create many Categories (one-to-many).
- A Task may reference one Category; category assignment is optional.

- Email addresses must be unique. Required fields and allowed status/priority values must be validated by the backend.
- Use Mongoose schema validation and timestamps. Add indexes where useful, including a unique index on email and an index on task owner/status.

System Architecture Diagram

A system architecture diagram is selected instead of a DFD. It shows how the browser-based frontend communicates with the Express backend, which uses Mongoose to access MongoDB Atlas. The optional external API box represents a possible future integration and is not required for the first version.

TASK MANAGEMENT SYSTEM - HIGH-LEVEL ARCHITECTURE



Authentication and validation are handled by the backend database credentials that stay on the server.

Request Flow

1. The user interacts with a page in the browser, such as the login page or task dashboard.
2. Frontend JavaScript sends an HTTP request to an Express API route using Fetch API.
3. The backend validates the input, checks authentication and task ownership, and calls the appropriate Mongoose model.
4. Mongoose reads or updates the relevant MongoDB Atlas document.

5. The server returns a JSON response. The front end displays the result or an understandable error message.

GitHub Repository Setup

Repository name: final-project- Group5TaskManagementSystem-BSIS2b

Repository URL: https://github.com/Ryuuu-Reaper/final-project-Group5TaskManagementSystem-BSIS2b

Personal Role Declaration

Member	Assigned role	Main responsibilities
Bea Roshan O. Pabilona	Project Manager + Documentation Officer	My assigned role is Project Manager, and I will also coordinate the project documentation because our group has five members. I will help the team understand the requirements, divide the work fairly, monitor deadlines, and communicate with our professor when clarification is needed. I will check that the proposal, diagrams, and repository structure are complete and consistent before submission. I also commit to listening to my teammates, helping resolve problems, and making sure everyone contributes to the project.
Eleanor Aimee O. Reantaso	Front-end Developer/ Front-End Manager	As the Front-End Manager, I am responsible for leading the development and design of the user interface for our Task Management System. I commit to creating a user-friendly, organized, and responsive interface that allows users to create, manage, and track their tasks easily. I will also coordinate with my groupmates to ensure that the front end works properly with the authentication and data persistence features. I will do my best to contribute my skills, communicate with the team, and complete my assigned tasks on time.

Christian A. Balauro	Backend Developer/ Back End-Manager	As the Back End-Manager my assigned role in this project is the back-end developer. I am responsible for handling the system's logic, processing data, and making sure the functions work properly. I commit to completing my assigned tasks, testing the system, and working with my groupmates to help make our project successful.
Simranjit S. Singh	Database Manager	As the database manager of our Task Management System project, I commit to organizing and maintaining the database, ensuring that all task and user information is stored accurately and securely. I will work closely with my groupmates, complete my responsibilities on time, and contribute to the success of our project.
Kar James M. Matias	GitHub Manager	As the GitHub Manager, my role is to manage our project repository, organize files, and ensure that all project updates are properly saved and uploaded to GitHub. I commit to keeping our repository organized, tracking changes, and making sure that every member can access the necessary project files. I will also help resolve GitHub-related issues and ensure that our task management system is properly documented and updated. Through this role, I will contribute to the successful completion of our project by maintaining an organized and reliable workspace for the team.
All members	Testing and debugging	Test the features assigned to each member, report reproducible issues, check fixes, and verify that integrated features work together.